

Solar Monitor

Developer Documentation

[Architecture](#) • [Services](#) • [Data Model](#) • [API](#) • [Inverter Integration](#)

dtitsolutions/solar-tracker — Version 1, June 2026

Contents

1. Introduction

Solar Monitor is a self-hosted, brand-agnostic dashboard for solar inverters. It polls one or more inverters on the local network, stores time-series readings, and serves a live web dashboard — with no dependency on the manufacturer's cloud. This document is for developers extending or operating the codebase; for plain installation see the Deployment & Operations Guide.

1.1 Design goals

- **Local-first:** read inverters directly over the LAN; the cloud is optional.
- **Fault isolation:** one OS process per inverter, so a single misbehaving device can't take down the fleet.
- **Brand-agnostic core:** inverter access sits behind a driver interface so new protocols slot in.
- **Operable:** Prometheus metrics, health endpoint, structured logs, and an in-app updater.

2. System Architecture

The system is a small set of containers orchestrated by Docker Compose. The browser only ever talks to the public web tier; the Python tier owns inverter access, persistence, and authentication.

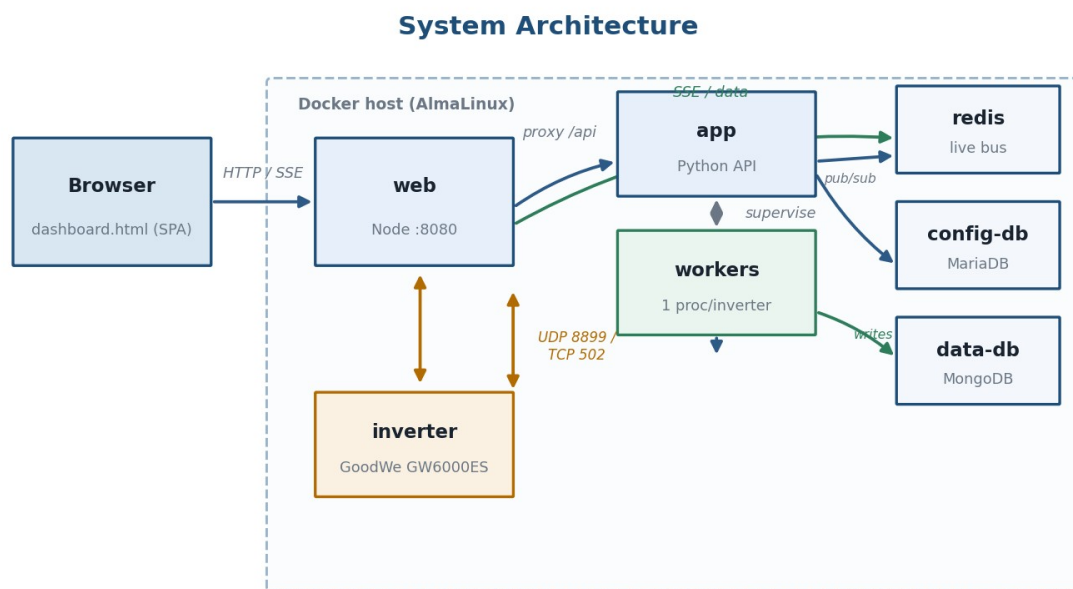


Figure 1 — Container topology and the connections between services.

2.1 Components

Service	Image / language	Responsibility
web	Node.js	Public HTTP :8080; serves dashboard.html, streams live data over SSE, proxies all other /api calls to the Python tier.
app	Python (stdlib http.server)	REST API, authentication, the inverter supervisor, and a mirror thread that pushes snapshots to Redis.
workers	Python (multiprocessing)	One child process per inverter: connect, poll, decode, persist, emit.
config-db	MariaDB	Users, roles, inverter definitions, settings.

Service	Image / language	Responsibility
data-db	MongoDB	Time-series readings.
redis	Redis	Low-latency live bus between the collector and the web tier.

2.2 Live data pipeline

Each poll cycle turns a raw inverter response into a normalized snapshot that is both stored (for history) and published (for live view).

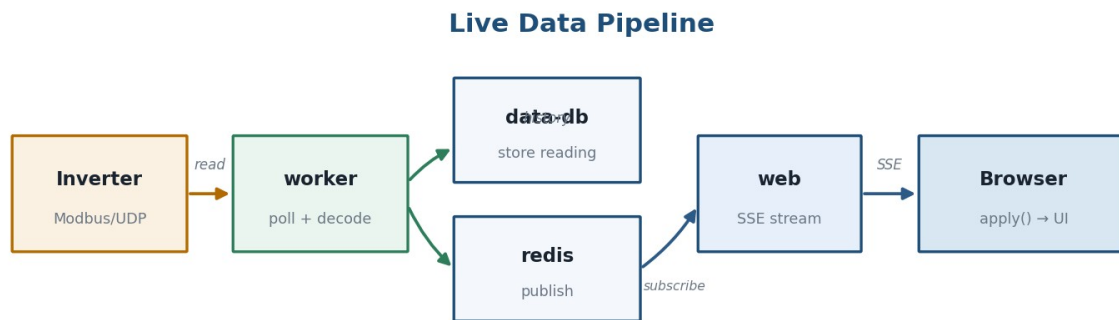


Figure 2 — From inverter read to browser render.

Path: a worker reads the inverter, decodes a snapshot, writes a reading document to MongoDB and publishes the snapshot to Redis; the web tier holds a Server-Sent-Events stream open to each browser and forwards new snapshots; the browser's `apply()` binds values to the UI.

2.3 Process & thread model (app container)

app Container — Process & Thread Model

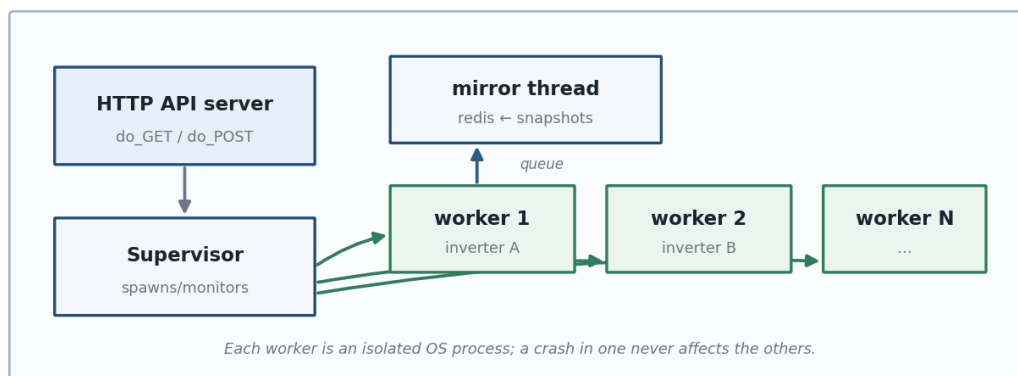


Figure 3 — The supervisor spawns isolated worker processes; the API server and mirror thread run alongside.

The supervisor (in `solar_monitor.py`) spawns and monitors one `workers.py` process per configured inverter. Workers hand snapshots back over a queue; a mirror thread relays the latest state to Redis. The HTTP API server runs in the main process. Because workers are separate OS processes, a crash or a wedged network read in one never blocks the others.

3. Technology Stack

Layer	Choice	Notes
Collector / API	Python 3 (stdlib)	http.server, multiprocessing, urllib — minimal dependencies
Inverter driver	goodwe (UDP) + Modbus TCP	pluggable per protocol
Web tier	Node.js	static serving + SSE + reverse proxy
Frontend	Vanilla JS + SVG	single dashboard.html, no build step
Config store	MariaDB	relational config/users
Data store	MongoDB	schema-light time series
Bus	Redis	pub/sub + last-state cache
Packaging	Docker Compose	one command up; image bakes the git SHA

4. Repository Layout

```

solar-tracker/
├── solar_monitor.py      # API, auth, supervisor, mirror, /metrics, /healthz
├── workers.py           # process-per-inverter: poll loop, daily() energy
├── config_store.py      # MariaDB: users, inverters, settings
├── data_store.py        # MongoDB: insert_reading, series()
├── logsetup.py          # structured logging + action/audit log
├── dashboard.html       # entire frontend (HTML/CSS/JS/SVG)
├── cli.py               # standalone single-inverter diagnostic/reader
├── lib/
│   └── inverter_goodwe.py # GoodWe driver: connect() + SUMMARY map
├── web/server.js        # Node web tier (SSE + proxy)
├── docker-compose.yml   # the stack
├── docker-compose.hostnet.yml # host-network override (LAN edge cases)
├── Dockerfile           # app image; ARG GIT_SHA -> ENV SM_DEPLOYED_SHA
├── scripts/             # install-docker.sh, build.sh, update.sh, units, probe.py
└── docs/               # this documentation

```

5. Service Reference

5.1 web/server.js (Node)

A thin public tier. It serves the dashboard, exposes the live endpoints from Redis, and proxies everything else to the Python API (which enforces its own auth).

- GET / → dashboard.html.
- GET /api/stream → Server-Sent Events; emits the current snapshot then each update from Redis.
- GET /api/data → the latest snapshot as JSON (one-shot).
- Catch-all → reverse-proxy to app:8080; the Python tier handles auth and returns 401 when needed.

5.2 solar_monitor.py (API + supervisor)

The heart of the Python tier. Built on the standard-library HTTP server.

Request handling & auth

- do_GET / do_POST dispatch on path. /api/login is handled before the auth gate; all other routes require a valid session.

- `self._authenticated()` validates the session cookie; `self._user()` returns the username; `config_store.is_admin(self._user())` gates admin-only routes.
- `self._json(code, payload)`, `self._read_json()`, `self._send(code, ctype, body)` are the response helpers.

Supervisor & mirror

- On startup it loads inverters from config and spawns a worker per inverter; it restarts a worker that dies and reaps it on shutdown.
- A mirror thread reads the newest per-inverter snapshot and writes combined live state to Redis for the web tier.

Update endpoints

- `check_for_update()` calls the GitHub commits API (token-aware), compares to the deployed SHA; `update_version()` returns the deployed SHA only (no internet); `request_update()` writes the trigger file.

5.3 workers.py (per-inverter process)

Each worker owns exactly one inverter. Pseudocode of the loop:

```
driver = lib.get_driver(brand)
inverter = loop.run_until_complete(driver.connect(ip, retries=3))
while not stop.is_set():
    raw = loop.run_until_complete(inverter.read_runtime_data())
    snap = normalize(raw)           # ppv, house_consumption, active_power, ...
    data_store.insert_reading(inv_id, snap) # history
    daily(snap)                    # today's energy + grid integration
    emit(snap, meta)               # -> supervisor queue -> redis
    stop.wait(poll_interval)
```

- `daily()` maintains today's generation/consumption (native counters) and integrates measured grid power into import/export; it resets at local midnight.
- `_seed_grid_today()` runs once on (re)start: it integrates today's already-logged readings so a mid-day restart doesn't zero the day's grid totals.

5.4 config_store.py (MariaDB) & data_store.py (MongoDB)

- `config_store`: users (hashed passwords, role), inverters (id, nickname, brand, ip), and settings; helpers like `is_admin()`, `set_password()`.
- `data_store`: `insert_reading(inverter_id, snap)` maps `active_power` → `grid_power_w` (+import/−export) and stores `recorded_at` in UTC; `series(inverter_id, since, until, limit)` returns rows oldest-first for history and seeding.

5.5 lib/inverter_goodwe.py (driver)

Exposes `async connect(ip, retries)` and a SUMMARY map of the sensor ids the dashboard cares about. The `connect` routine tries an explicit family hint (`SM_GOODWE_FAMILY`), then auto-detect, then a sweep of ES/ET/EH/DT/BP/EM — making GoodWe's flaky discovery reliable. See Section 7 for the Modbus-TCP path.

6. Data Model

6.1 Reading snapshot (normalized keys)

The canonical keys produced by a worker and consumed by the dashboard:

Key	Meaning	Sign / unit
ppv	PV / solar power	W
house_consumption	House load	W
active_power	Grid power	+ import / – export, W
pbattery1	Battery power	+ charge / – discharge, W
battery_soc	Battery state of charge	%
e_day	Generation today	kWh
e_load_day	Consumption today	kWh
e_total	Lifetime generation	kWh
temperature	Inverter temperature	°C

6.2 MongoDB reading document

```
{ inverter_id: <id>,
  recorded_at: <UTC ISO>,           // indexed for range queries
  grid_power_w: <+imp/-exp>,         // derived from active_power
  solar_power_w, load_power_w, battery_power_w, battery_soc, ... }
```

6.3 Redis live state

The mirror thread keeps the newest combined snapshot under a well-known key and publishes updates on a channel the web tier subscribes to. The web tier never reads the databases directly.

7. HTTP API Reference

All routes are served by the Python tier (proxied through the web tier). JSON in/out. Admin-only routes return 403 for non-admins; unauthenticated requests return 401.

Method & path	Auth	Purpose
POST /api/login	public	create a session
POST /api/logout	user	end the session
GET /api/me	user	current user + role
GET /api/data	user	latest snapshot
GET /api/stream	user	SSE live stream
GET /api/history	user	windowed time-series for the chart
GET /api/inverters	user	list inverters
POST /api/inverters	admin	add / edit / remove an inverter
GET/POST /api/users...	admin	user administration
GET /api/settings	user	server settings
GET /api/update/check	admin	compare deployed vs GitHub
GET /api/update/version	admin	deployed SHA (no internet)
POST /api/update/apply	admin	trigger a self-update
GET /metrics	open*	Prometheus metrics
GET /healthz	open*	health probe

* metrics/health are intended for an internal network; restrict at the reverse proxy if exposed.

8. Inverter Integration

The most important — and most device-specific — part of the system. GoodWe dongles speak one of two local protocols depending on generation.

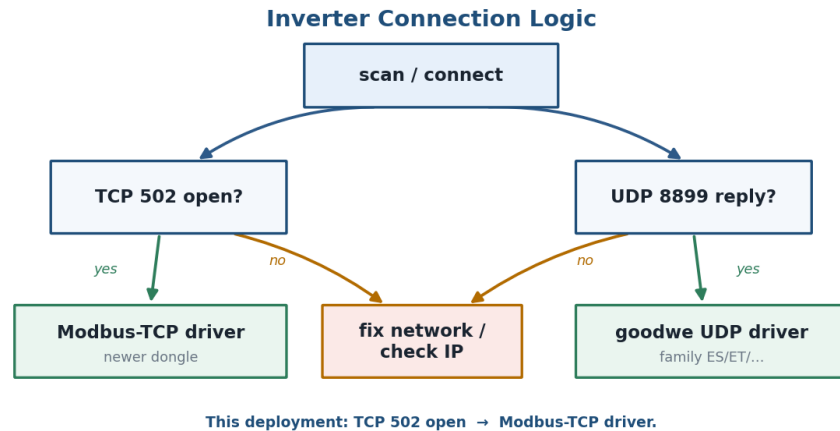


Figure 4 — Choosing the driver from what the dongle exposes.

8.1 Protocols

Protocol	Transport	Framing	Library / code
GoodWe UDP	UDP 8899	AA55 (ES) or Modbus-RTU-over-UDP (ET)	goodwe package
Modbus TCP	TCP 502	Modbus TCP (MBAP + PDU)	custom reader (cli.py / planned driver)

8.2 connect() strategy (UDP)

```

order = [hint] if hint else []
order += [None] # auto-discovery
order += ['ES', 'ET', 'EH', 'DT', 'BP', 'EM'] # ES first (GW####ES)
for fam in order:
    try: return goodwe.connect(host, family=fam, timeout, retries)
    except: keep trying

```

8.3 Modbus TCP (newer dongles)

Newer dongles — the kind the SEMS+ app connects to locally — answer on TCP 502 with Modbus. A read is an MBAP header + PDU (function 3). Device info lives at 0x88B8 (model, serial at byte offset 6, rated power); running data starts at 0x891C; battery SoC is in the BMS block (0x908F). The device-info serial provides a built-in decode check.

```

# Modbus-TCP read (function 3), no external deps
pdu = struct.pack('>BHH', 0x03, start, qty)
adu = struct.pack('>HHHB', 1, 0, len(pdu)+1, unit) + pdu # unit 0xF7
# response: MBAP(7) + func(1) + bytecount(1) + data

```

8.4 Adding a driver

Drivers live in `lib/` and are resolved by `lib.get_driver(brand)`. A driver provides an `async connect(ip, retries)` returning an object with `read_runtime_data()`, `sensors()`, `model_name` and

`serial_number`. To add Modbus-TCP support, implement a driver that wraps the reads above and normalizes to the keys in Section 6.1.

9. Energy & Sign Conventions

- **Grid** `active_power`: positive = importing from grid, negative = exporting.
- **Battery** `pbattery1`: positive = charging, negative = discharging.
- **Daily import/export** are integrated from measured grid power (not the inverter's native daily counters, which are unreliable on this ES unit), and seeded from logged data on restart.
- **Generation/consumption today** use the inverter's native `e_day` / `e_load_day` counters.

10. Frontend Architecture

The whole UI is one `dashboard.html` — HTML, CSS and vanilla JS, no build step. It is a single-page app with an admin shell (sidebar + topbar + content).

- **Pages:** `showPage(id)` toggles `.page.active` (Dashboard, History, Fleet, Inverter Info, Settings).
- **Live binding:** `apply(snapshot)` updates KPI cards, the flow diagram, and the 3D house from one data object so the views never disagree.
- **Flow views:** a toggle switches the circles diagram and the isometric 3D house; the choice is remembered in `localStorage`.
- **Updates UI:** an admin-only Updates panel and a global banner call the update endpoints, with a browser-side GitHub fallback.
- **Verification:** the frontend is checked with `node --check` and a DOM-shim harness that loads the script and asserts no init errors.

11. Authentication & Roles

Sessions are cookie-based and validated on every request by the Python tier. Two roles: admin (full control, including users, inverters and updates) and user (read the dashboard). The default admin/admin must be changed on first login. Non-admin UI is hidden via a `body.role-user` CSS class and enforced server-side.

12. Build, Deploy & Self-Update

The image bakes the current commit via Dockerfile `ARG GIT_SHA → ENV SM_DEPLOYED_SHA`, so the running app knows its own version. `build.sh` passes `GIT_SHA=$(git rev-parse HEAD)`.

Build & Self-Update Flow

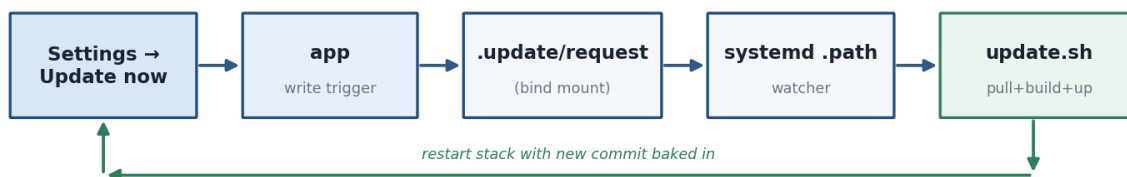


Figure 5 — A container can't rebuild itself, so the app signals a host-side watcher.

On "Update now", the app writes `.update/request` (a bind-mounted directory). A systemd `.path` unit watches that file and runs `update.sh: git pull → docker compose build → up -d`, then records the new SHA and clears the trigger. No Docker socket is exposed to the container.

13. Observability

- `/metrics` — Prometheus counters/gauges (poll success/failure, connection state, last-read age).
- `/healthz` — liveness/health for probes and uptime checks.
- Structured logs via `logsetup.py`, including an action/audit log for admin operations (logins, user changes, update requests).

14. Testing & Verification

The sandbox used to build this has no network, Docker or real databases, so verification leans on fast static and stand-in checks:

- `python -m py_compile` for every Python module.
- `node --check` on the extracted dashboard script.
- A DOM-shim harness that loads the dashboard JS against mock elements/fetch and asserts initialization runs without errors and dropdowns populate.
- In-memory stand-ins for `config_store/data_store`, and unit tests for the tricky logic (grid-seeding integration, SHA comparison, Modbus framing/decoding).

15. Extending the System

- **New inverter brand/protocol:** add a driver under `lib/` and register it with `lib.get_driver`; normalize to the Section 6.1 keys.
- **New dashboard view:** add a `.page`, a nav item with `data-page`, and bind data in `apply()`.
- **New metric:** emit it from the worker snapshot and expose it on `/metrics`.
- **Run from source:** use `cli.py` to validate inverter access before wiring a new driver into the stack.

16. Appendix — Environment Variables

Variable	Default	Purpose
<code>SM_GOODWE_FAMILY</code>	(empty)	Force a UDP protocol family (e.g. ES)
<code>SM_GOODWE_TIMEOUT</code>	2	Per-read timeout (s)
<code>SM_GITHUB_REPO</code>	<code>dtitsolutions/solar-tracker</code>	Repo the updater checks
<code>SM_GITHUB_BRANCH</code>	<code>main</code>	Branch tracked
<code>SM_GITHUB_TOKEN</code>	(empty)	Token for a private repo
<code>SM_DEPLOYED_SHA</code>	(build arg)	Commit baked into the image
<code>SM_UPDATE_TRIGGER</code>	<code>/app/.update/request</code>	Self-update trigger file
<code>SM_OFFLINE_TIMEOUT_SECONDS</code>	120	Mark an inverter offline after silence

— *End of developer documentation* —